
Tema 1

Manejo de Ficheros

Acceso a Datos



salesianos
LOS BOSCOS · LOGROÑO

Jesús Allona del Río @ Salesianos Los Boscos - Logroño

Introducción

En este tema estudiaremos algunas de las posibles formas para **trabajar con ficheros**. Aunque no son todas las posibles, estas soluciones pueden considerarse una referencia dentro de las soluciones actuales respecto al almacenamiento de datos en ficheros.

Haremos uso del lenguaje **C#** de Microsoft .NET, destacando especialmente el espacio de nombres **System.IO**, que contiene tipos que permiten leer y escribir en los archivos y secuencias de datos, así como tipos que proporcionan compatibilidad básica con los archivos y directorios.

Un fichero o archivo es un conjunto de bits almacenado en un dispositivo, como por ejemplo un disco duro. Una de las ventajas de usar ficheros es que éstos no son volátiles. Los ficheros tienen un nombre y se ubican en directorios o carpetas, el nombre debe ser único en ese directorio. Por convención, cuentan con diferentes extensiones que por lo general suelen ser tres letras (PDF, DOC, GIF, ...) y nos permiten saber el tipo de archivo.

Clases asociadas

Antes de ver las clases que leen y escriben datos en ficheros, vamos a manejar las siguientes clases:

- Clase **File** (FileInfo)
 - File es una clase perteneciente al “espacio de nombres” System.IO
 - Representa un archivo dentro de un sistema de ficheros.
 - Proporciona métodos para crear, copiar, eliminar, mover y abrir un archivo.
- Clase **Directory** (DirectoryInfo)
 - Directory es una clase perteneciente al “espacio de nombres” System.IO
 - Representa un directorio (carpeta) dentro de un sistema de ficheros.
 - Proporciona métodos para crear, mover y enumerar archivos en directorios (subdirectorios)

Clase File

La clase File proporciona métodos estáticos para crear, copiar, eliminar, mover y abrir un solo archivo, y contribuye a la creación de objetos FileStream.

La clase FileInfo nos permitiría hacer lo mismo pero mediante una instancia del mismo.

Dado que todos los métodos de File son estáticos, podría ser más eficaz usar un método File en lugar de un método de una instancia de FileInfo si desea realizar una sola acción.

Los métodos estáticos de la clase File realizan comprobaciones de seguridad en todos los métodos. Si va a reutilizar un objeto varias veces, considere la posibilidad de usar en su lugar una instancia de FileInfo, porque la comprobación de seguridad no siempre será necesaria.

Algunos métodos de File

Documentación File

Estos serían algunos de los principales métodos de la clase File:

- **Create**(String): *Crea o sobrescribe un archivo en la ruta de acceso especificada.*
- **Copy**(String, String): *Copia un archivo existente en un archivo nuevo. No se permite sobrescribir un archivo del mismo nombre.*
- **Delete**(String): *Elimina el archivo especificado.*
- **Move**(String, String): *Mueve un archivo especificado a una nueva ubicación, proporcionando la opción para indicar un nuevo nombre de archivo.*
- **Open**(String, FileMode): *Abre un elemento FileStream en la ruta de acceso especificada con acceso de lectura y escritura y sin posibilidad de compartir.*
- **OpenRead**(String): *Abre un archivo existente para lectura.*
- **OpenText**(String): *Abre un archivo de texto existente con codificación UTF-8 para lectura.*

Ejemplo File

```
FileStream fs = File.Create(@"C:\Users\yoMismo\Desktop\prueba.txt");  
fs.Close();
```

* *¿Por qué hacemos el fs.Close()?*

Clase Directory

La clase Directory proporciona métodos estáticos para crear, mover, borrar directorios, y enumerar archivos en directorios y subdirectorios.

La clase DirectoryInfo nos permitiría hacer lo mismo pero mediante una instancia del mismo.

Dado que todos los métodos de Directory son estáticos, podría ser más eficaz usar un método Directory en lugar de un método de una instancia de DirectoryInfo si desea realizar una sola acción.

Los métodos estáticos de la clase Directory realizan comprobaciones de seguridad en todos los métodos. Si va a reutilizar un objeto varias veces, considere la posibilidad de usar en su lugar una instancia de DirectoryInfo, porque la comprobación de seguridad no siempre será necesaria.

Algunos métodos de Directory

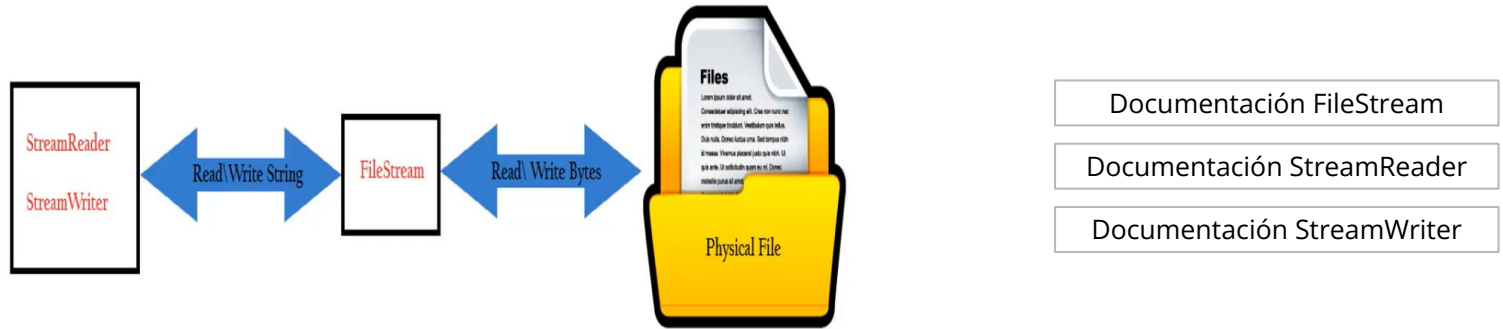
Estos serían algunos de los principales métodos de la clase Directory:

- **CreateDirectory**(String): *Crea todos los directorios y subdirectorios en la ruta de acceso especificada, a menos que ya existan.*
- **Delete**(String): *Elimina un directorio vacío de una ruta de acceso especificada.*
- **Move**(String, String): *Mueve un archivo o directorio y su contenido a una nueva ubicación.*
- **GetFiles**(String): *Devuelve los nombres de archivo (con sus rutas de acceso) del directorio especificado.*
- **GetDirectories**(String): *Devuelve los nombres de los subdirectorios (con sus rutas de acceso) del directorio especificado.*
- **GetParent**(String): *Recupera el directorio principal de la ruta especificada, incluidas tanto las rutas de acceso absolutas como las relativas.*

Ejemplo Directory

```
Directory.CreateDirectory(@"C:\MiDirectorio");
```

Flujos (Streams)

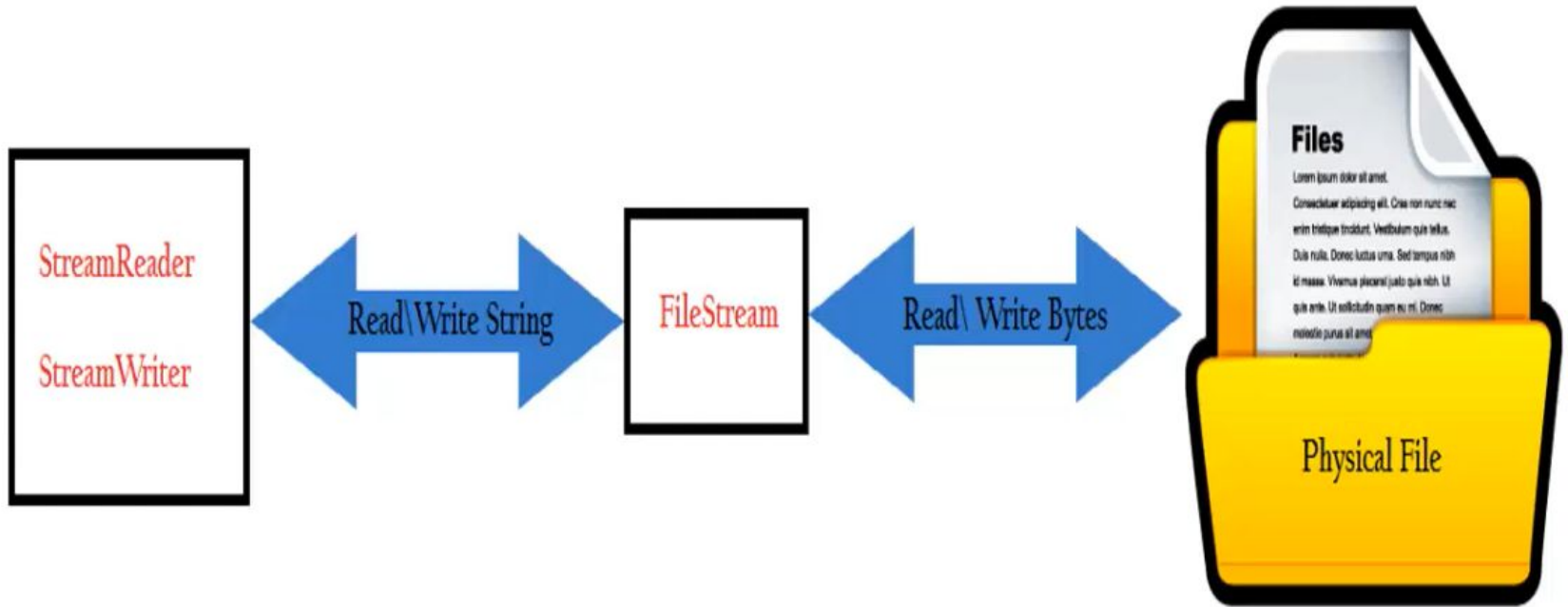


FileStream: Proporciona un Stream para un archivo, lo que permite operaciones de lectura y escritura.

StreamReader: Implementa un `TextReader` que lee los caracteres de una secuencia de bytes en una codificación determinada.

StreamWriter: Implementa un `TextWriter` para escribir los caracteres de una secuencia en una codificación determinada.

Flujos (Streams)



FileStream

```
FileStream fs = new FileStream(string fichero, FileMode mode, FileAccess access)
```

fichero (path): Fichero (ruta) que vamos a abrir.

FileMode: Modo de apertura del fichero.

- Open
- OpenOrCreate
- Create
- Append
- Truncate

FileAccess: Tipo de acceso al fichero.

- Read
- Write
- ReadWrite

Algunos métodos de FileStream:

- Write
- WriteByte
- Read
- ReadByte
- Seek
- Close
- ...

StreamReader

```
StreamReader sr = new StreamReader(FileStream fs)
```

FileStream: FileStream (descrito en la diapositiva anterior) con el que vamos a trabajar.

Algunos métodos de StreamReader:

- Read
- ReadToEnd
- ReadLine
- Close
- Peek
- ...

StreamWriter

```
StreamWriter sw = new StreamWriter(FileStream fs)
```

FileWriter: FileStream (descrito en la diapositiva anterior) con el que vamos a trabajar.

Algunos métodos de StreamWriter:

- Write
- WriteLine
- Close
- ...

```
using (FileStream fs = new FileStream(fichero, FileMode.Create, FileAccess.Write))  
using (StreamWriter sw = new StreamWriter(fs))  
{  
    sw.Write("Nuevo contenido");  
}
```



Darí­a exactamente igual utilizar un método que otro



```
FileStream fs = new FileStream(fichero, FileMode.Create, FileAccess.Write);  
StreamWriter sw = new StreamWriter(fs);  
  
sw.Write("Nuevo contenido");  
  
sw.Close();  
fs.Close();
```

Ejemplo uso StreamWriter

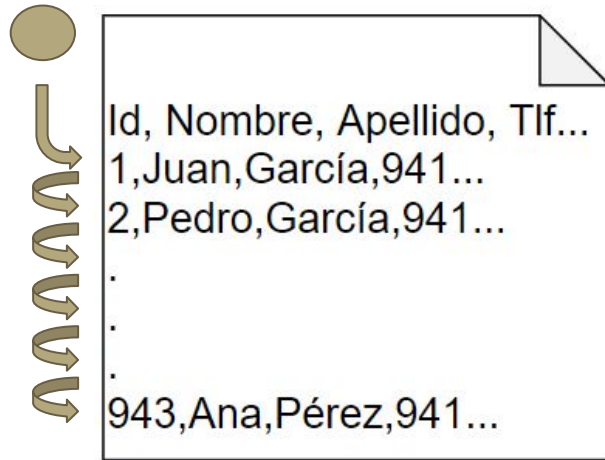
Acceso Aleatorio a Fichero

Hasta ahora hemos trabajado con un acceso secuencial a ficheros, más orientado a líneas completas, el cual no resulta demasiado adecuado para según qué aplicaciones, en las que tengamos que trabajar de una manera **más eficiente** con el contenido de un fichero.

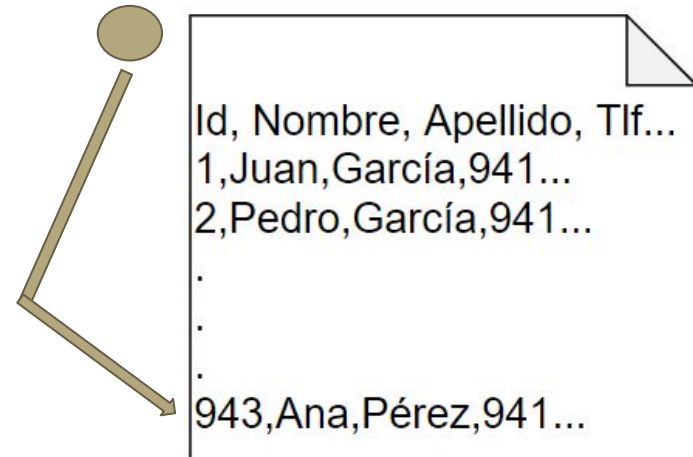
Para este tipo de acceso, normalmente la información se almacena **organizada en registros de tamaño fijo**, con el objetivo de poderla acceder de una manera **más directa (aleatoria)** y sin la necesidad de recorrer todo el fichero línea a línea hasta llegar a la información deseada.

Acceso Aleatorio vs Acceso Secuencial

Acceso Secuencial

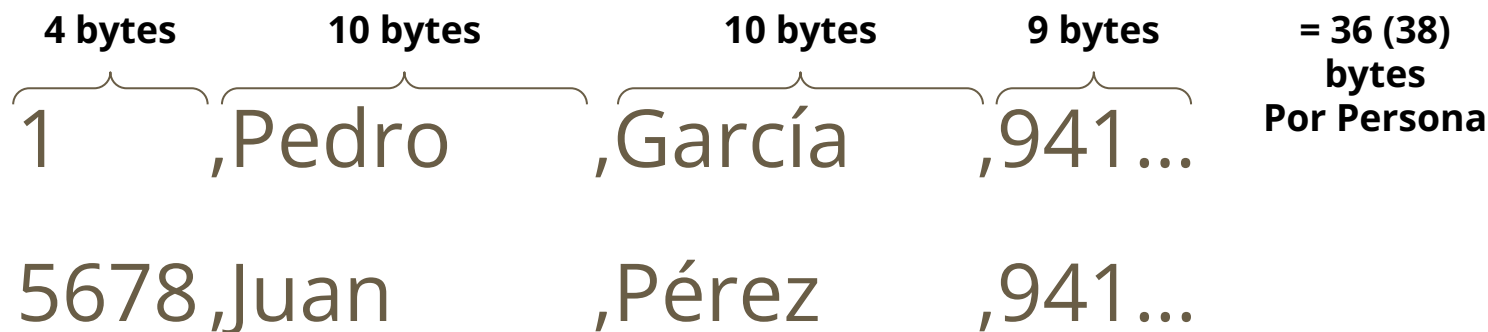


Acceso Aleatorio



Organización de Información para Acceso Aleatorio

Cómo hemos dicho anteriormente, para el tipo de acceso aleatorio, normalmente la información se almacena **organizada en registros de tamaño fijo**, con el objetivo de poderla acceder de una manera **más directa (aleatoria)** y sin la necesidad de recorrer todo el fichero línea a línea hasta llegar a la información deseada.



Método Acceso Aleatorio

Cuando queramos acceder a un fichero de manera aleatoria, deberemos hacer uso del método "**Seek**" de **FileStream**.

Funcionamiento del método **Seek** (long **salto**, SeekOrigin **referencia**)

- Salto: posición del fichero a la que queremos saltar
 - Positivo (6): salta 6 bytes hacia adelante
 - Negativo (-6): salta 6 bytes hacia atrás
- Referencia: el punto desde el que queremos que se haga el *salto*
 - SeekOrigin.Begin: desde el comienzo del fichero
 - SeekOrigin.Current: desde la posición actual
 - SeekOrigin.End: desde el final del fichero

```
using (FileStream fs = new FileStream(fichero, FileMode.Open, FileAccess.Read))  
{  
    fs.Seek(salto, SeekOrigin.Begin);  
}
```

De igual modo, podemos saber en qué posición del fichero nos encontramos, consultando la propiedad "Position", así como la longitud del fichero, mirando su propiedad "Length".

Ejemplos Acceso Aleatorio

4 bytes 10 bytes 10 bytes 9 bytes = 33 bytes
Por Persona

1 Pedro García 941...

5678 Juan Pérez 941...

Sin salto de línea (t

Sin salto de línea (todo seguido)

- | | | | | |
|----|------|-------|----------|--|
| 1. | 1 | Pedro | García → | .Seek(14, SeekOrigin.Begin) |
| 2. | 1 | Pedro | García → | .Seek(-10, SeekOrigin.Current)
.Seek(4, SeekOrigin.Begin) |
| 3. | 1 | Pedro | García → | .Seek(33, SeekOrigin.Current) |
| | 5678 | Juan | Pérez | .Seek(37, SeekOrigin.Begin)
.Seek(-29, SeekOrigin.End) |

Lectura / Escritura Recomendada para Aleatorio

Para leer y escribir en un fichero al cual estamos accediendo de manera aleatoria, teniendo en cuenta que la información se almacena **organizada en registros de tamaño fijo**, sería recomendable usar los métodos “**Read**” y “**Write**” de **FileStream**.

Funcionamiento del método **Read** (byte[] arrayLectura, int offset, int numBytesAleer)

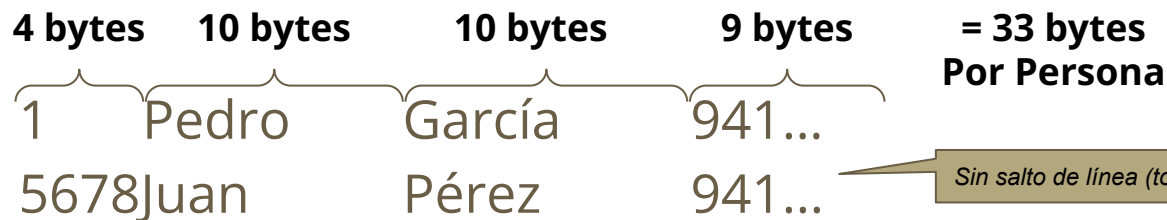
- arrayLectura: array de destino donde se almacenarán los bytes leídos
- offset: desplazamiento en arrayLectura para comenzar a almacenar (normalmente es 0)
- numBytesAleer: número de bytes que se pretenden leer

Funcionamiento del método **Write**(byte[] arrayEscrit, int offset, int numBytesAescribir)

- arrayEscritura: array de origen desde donde se escribirán los bytes
- offset: desplazamiento en arrayEscrit para comenzar a leer (normalmente es 0)
- numBytesAescribir: número de bytes que se pretenden escribir

Encoding.ASCII.GetBytes("asdf") // Encoding.ASCII.GetString(arrayBytes)

Ejemplos Acceso Aleatorio



- 1. Leer apellido de 1** →
`fs.Seek(14, SeekOrigin.Begin)`
`byte[] lectura = new byte[10];`
`fs.Read(lectura, 0, lectura.Length);`
- 2. Leer nombre de 1** →
`fs.Seek(-20, SeekOrigin.Current)`
`byte[] lectura = new byte[10];`
`fs.Read(lectura, 0, lectura.Length);`
`fs.Seek(4, SeekOrigin.Begin)`
`byte[] lectura = new byte[10];`
`fs.Read(lectura, 0, lectura.Length);`
- 3. Leer Id de Juan** →
`.Seek(19, SeekOrigin.Current)`
`byte[] lectura = new byte[4];`
`fs.Read(lectura, 0, lectura.Length);`
`.Seek(-33, SeekOrigin.End)`
`byte[] lectura = new byte[4];`
`fs.Read(lectura, 0, lectura.Length);`

Fin